

decommissioned or modified without the necessary sign-off. The documentation on building servers dealt only with the technical issues. That's a reasonable approach, but it wasn't going to help me. I felt that I needed notes that walked me through the process from beginning to end, in the order in which I had to perform tasks.

So I started compiling my own notes. I wasn't rewriting the existing build instructions, though occasionally I did add a clarifying note or two. It was more like writing a recipe. The details aren't all that important.

I found that navigating the paper-work was the major challenge. I could build three or four VMs concurrently, so that part went smoothly and fairly quickly. But there were often long pauses while I waited for approvals from other parts of the change-management system. Then, requests began coming in to build more VMs.

Within a few weeks, I felt on top of the situation. My notes were reasonably good. I was asking fewer questions. Things had settled down.

I went to see my boss, because it seemed I had run out of work. I'm prepared to tolerate a wide spectrum of working conditions, but I had said repeatedly that he would have to pay me a huge amount of money if I was going to be bored.

My boss had noticed that I had been making notes. He wanted to know if I could turn my notes into documentation that could be used by others.

I eagerly accepted this project.

It seemed to me that the major issue was going to be that my notes were incomplete: I had not yet encountered every scenario when it came to building VMs. If I tidied up the notes and delivered them, I could expect that a few days later I would need to update them. I would be delivering updates almost constantly. There had to be a better way.

I decided that I could kill two birds with one stone. Notes that I keep for myself, I usually write in plain text files. They form part of a 'My Help System'. I don't spend time on appearances. But if I was planning to make notes for others, then presentation became a little more important. I might have been persuaded to write in Microsoft Word or LibreOffice—but I preferred to avoid these.

I decided to build a website to serve the documentation. Any time someone went to the website, it would reflect the current state of my documentation. If they visited a few days later, all recent updates would be available. The other advantage for me would be that I could write HTML and use my favourite editor (emacs).

I had been lucky. Most of the staff at Acme Corporation ran XP on their PCs. Kevin was my soul brother: he ran Linux (Ubuntu) with a copy of XP running inside VMware Player. This makes eminently more sense to me than the alternatives.

When I worked at IBM, everyone was issued with

laptops running XP. I tried using Putty, but I didn't achieve maximum productivity until I installed VirtualBox running Fedora. I dedicated the large second monitor to Fedora, and kept the laptop's primary screen for XP. Needless to say, I spent as much time as possible in Fedora, only venturing out when absolutely necessary. But still, it was not ideal. Kevin's set-up was much better. Very kindly, he bequeathed me his PC in his absence.

So building a website on Kevin's PC presented no insurmountable difficulties. My first thought was Apache. I'd built websites using Apache in the past. I poked around Kevin's machine. Apache was already running.

I guess my next steps reflect my view of etiquette. It seems to me that there are some things you just don't do. You like to give a borrowed item back to its owner in good condition. Obviously, there were signs of me on his machine. Kevin had created an account for me, and I guess we both expected that I could create whatever files and directories I wanted under my home directory. But, in my view, messing with his website was perhaps a bridge too far.

I suppose I could have installed another instance of Apache. But Apache is pretty fat. For what I had in mind, it just seemed like overkill. I mean, we were expecting, at the most, one or two occasional visits to this website. Again, I felt there had to be a better way.

I may have been lucky on the day I searched. I discovered SimpleHTTPServer, a Python script that just does the job.

At this point, you might be inclined to point out that I have failed to deliver on the title of this article. Guilty as charged—but I'm going to make up for it now.

Warning

Be aware that you are about to fire up a Web server. As trivial as that may be, there may still be some risks involved. Ideally, your machine will be behind a firewall, and perhaps even the machine itself will be protected by something like *iptables*. If not, consider carefully whether you want to proceed. There is another way to provide some protection. I'll discuss that a bit later (in 'Deeper wrinkles').

The other side of the protection coin is this: others may not be able to access your server because your machine is so tightly locked down that nothing gets in or out. This seems to be the prevailing philosophy with recent distributions. Although I understand the rationale, I still find myself stubbing my toe. This week, I downloaded a VM of Fedora 16. I usually don't want to actually run on the 'console' of my VMs; I'm much happier SSHing in. But before I can do that, I have to turn on the SSH daemon and turn off *iptables*. (I also turn off SELinux, but that may simply be out of superstition.) You